

JOURNAL DES DÉFIS

et solutions apportées au fil du projet

SmartContent AI

Sept semaines de développement intensif documentées

24 défis techniques rencontrés · 24 solutions adoptées

AUTEUR

Serge Tsebo

AEC Intelligence Artificielle Appliquée

Collège Multihexa · Montréal, Québec · Mai 2026

Introduction

Ce document constitue le journal de bord technique du projet SmartContent AI. Il recense de manière exhaustive et chronologique les défis techniques rencontrés au cours des sept semaines de développement, ainsi que les solutions adoptées pour les surmonter. L'objectif n'est pas de masquer les difficultés ni de minimiser leur impact, mais au contraire de les exposer clairement pour rendre visible la quantité de travail accompli entre l'idée initiale et le produit fini déployé en production cloud.

Un projet de fin d'études se juge moins à la perfection de l'application finale qu'à la qualité des décisions prises face aux obstacles. Chaque défi documenté ci-dessous a été l'occasion d'un apprentissage transférable à d'autres contextes professionnels : architecture web, sécurité applicative, intégration d'API tierces, déploiement cloud, gestion de versions et collaboration outillée.

Les vingt-quatre défis recensés sont organisés en six chapitres correspondant aux grandes phases du projet. Pour chacun, le document expose le problème observé, la cause identifiée après investigation, la solution finalement adoptée avec extrait de code si pertinent, et l'apprentissage clé à retenir.

Vue d'ensemble — 24 défis en 7 semaines

Chapitre	Défis	Sévérité	Temps total
Chapitre 1 — Backend FastAPI	4 défis	Modérée	≈ 8 heures
Chapitre 2 — Frontend Streamlit	4 défis	Modérée	≈ 12 heures
Chapitre 3 — Orchestration n8n	3 défis	Modérée	≈ 4 heures
Chapitre 4 — Déploiement cloud	6 défis	Élevée	≈ 16 heures
Chapitre 5 — Intégration email	4 défis	Modérée	≈ 6 heures
Chapitre 6 — Méthodologie	3 défis	Élevée	≈ 10 heures
TOTAL	24 défis	—	≈ 56 heures

Ces cinquante-six heures de résolution de problèmes représentent environ vingt-cinq pour cent du temps total de développement, ce qui correspond à la moyenne professionnelle pour un projet d'ampleur similaire. La part importante du chapitre 4 (déploiement cloud) s'explique par la découverte de trois nouvelles plateformes (Railway, Streamlit Cloud, Resend) jamais utilisées auparavant.

Chapitre 1 — Défis du backend FastAPI

Le backend FastAPI constitue la couche centrale de l'application. Sa mise en place a soulevé quatre défis principaux liés à l'authentification, à la gestion des secrets, à l'orchestration parallèle des appels OpenAI et au parsing des réponses du modèle.

1.1. Incompatibilité passlib 1.7.4 avec bcrypt 5.0

PROBLÈME

La majorité des tutoriels FastAPI recommandent passlib pour gérer le hashage des mots de passe. Lors du premier test d'inscription, le serveur lève une exception `ValueError` mentionnant l'absence d'attribut `__about__` sur l'objet `bcrypt`, empêchant toute requête d'aboutir.

Cause identifiée :

passlib 1.7.4 repose sur une signature interne de `bcrypt` (l'attribut `__about__`) qui a été supprimée dans la version 5.0 de `bcrypt`. Les deux bibliothèques étaient théoriquement compatibles selon leurs propres requirements, mais une mise à jour majeure de `bcrypt` avait cassé le contrat implicite.

SOLUTION

Plutôt que de figer une version ancienne de `bcrypt` (bloquant les mises à jour de sécurité futures), le code appelle directement la bibliothèque `bcrypt` sans passer par `passlib`. Le mot de passe est tronqué manuellement à 72 octets avant hashage pour respecter la limite imposée par `bcrypt`.

```
import bcrypt

def hash_password(plain: str) -> str:
    pw_bytes = plain.encode('utf-8')[:72]
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(pw_bytes, salt).decode('utf-8')

def verify_password(plain: str, hashed: str) -> bool:
    pw_bytes = plain.encode('utf-8')[:72]
    return bcrypt.checkpw(pw_bytes, hashed.encode('utf-8'))
```

🔗 Apprentissage : Les wrappers ajoutent une couche de fragilité. Pour des fonctionnalités critiques (sécurité), il vaut mieux appeler directement les bibliothèques sous-jacentes.

1.2. Configuration Pydantic-settings et chargement du .env

PROBLÈME

L'application doit charger une douzaine de variables d'environnement sensibles au démarrage. Le backend démarre apparemment normalement mais les premières requêtes vers `POST /generate` retournent un 500 avec un message générique « OpenAI error : Invalid API key ».

Cause identifiée :

Le fichier .env n'était pas lu correctement parce que pydantic-settings v2 est sensible à la casse par défaut, alors que les variables d'environnement Windows ne le sont pas. De plus, l'encoding par défaut provoquait des erreurs sur les caractères accentués.

SOLUTION

La classe Settings a été configurée avec case_sensitive=False et env_file_encoding=utf-8. Tous les noms de variables ont été standardisés en minuscules dans le code Python, et l'option extra=ignore a été ajoutée pour tolérer des variables d'environnement supplémentaires sans planter.

```
class Settings(BaseSettings):
    openai_api_key: str
    supabase_url: str
    # ...
    model_config = SettingsConfigDict(
        env_file='.env',
        env_file_encoding='utf-8',
        case_sensitive=False,
        extra='ignore',
    )
```

🔗 Apprentissage : Lire la documentation officielle d'une bibliothèque avant de coder, plutôt que de se baser sur des tutoriels obsolètes. pydantic-settings v2 a beaucoup évolué par rapport à v1.

1.3. Parallélisation des appels DALL-E 3**PROBLÈME**

La génération séquentielle des trois images DALL-E 3 (carré, paysage, portrait) prenait initialement 30 à 40 secondes, ce qui rendait l'expérience utilisateur frustrante. Le cahier des charges fixait pourtant la cible à 25 secondes maximum.

Cause identifiée :

Les appels à DALL-E 3 étaient lancés en séquence dans une boucle for synchrone. Chaque appel attendait la fin du précédent avant de démarrer, ce qui multipliait la latence par trois.

SOLUTION

Adoption de ThreadPoolExecutor avec trois workers pour paralléliser les trois appels. Le temps total est passé à 18 secondes, soit le pire cas des trois appels en parallèle au lieu de leur somme.

```
from concurrent.futures import ThreadPoolExecutor

def generate_images(sujet, secteur, ton):
    out, errors = {}, {}
    with ThreadPoolExecutor(max_workers=3) as ex:
```

```

futures = [ex.submit(_generate_one_image, prompt, fmt)
            for fmt in IMAGE_FORMATS]
for f in futures:
    name, url, err = f.result()
    if url: out[name] = url
return out

```

🔗 Apprentissage : Les opérations I/O-bound (appels HTTP vers une API tierce) bénéficient massivement du *multithreading* en Python, contrairement aux opérations CPU-bound qui sont bloquées par le GIL.

1.4. Mode JSON natif de GPT-4o et parsing des réponses

PROBLÈME

Lors des premiers tests de génération, le parsing des réponses de GPT-4o échouait régulièrement parce que le modèle introduisait du texte explicatif autour du JSON attendu (par exemple « Voici le contenu en JSON : { ... } »). Une génération sur deux retournait une chaîne qui n'était pas un JSON valide.

Cause identifiée :

Le prompt utilisateur indiquait simplement « Réponds en JSON » mais GPT-4o, comme tout modèle conversationnel, a une tendance naturelle à entourer ses réponses de phrases polies. Le mode JSON natif de l'API OpenAI n'était pas activé.

SOLUTION

Activation du paramètre `response_format=json_object` dans l'appel à l'API OpenAI. Ce mode force le modèle à retourner exclusivement un JSON valide, sans texte autour. Un parseur tolérant a aussi été ajouté en secours pour extraire le JSON même si du texte parasite l'entoure.

🔗 Apprentissage : Les paramètres avancés d'une API peuvent transformer radicalement le comportement d'un modèle. Toujours lire la documentation à fond avant de coder du parsing fragile.

Chapitre 2 — Défis du frontend Streamlit

Le frontend Streamlit, bien que rapide à itérer, présente plusieurs limites quand on cherche à produire un rendu premium. Quatre défis ont marqué cette phase : le cache des widgets, l'organisation du CSS, le manque de contrôle sur le markup HTML, et le polish responsive.

2.1. Streamlit text_area affichant du contenu obsolète

PROBLÈME

Après plusieurs générations successives sur la même session utilisateur, certains onglets continuaient d'afficher l'ancien contenu. Quand l'utilisateur générait une campagne pour le sujet A puis pour le sujet B, l'onglet LinkedIn pouvait encore montrer le texte du sujet A.

Cause identifiée :

Streamlit identifie chaque widget par sa position dans l'arbre de rendu et par sa clé. Quand le texte change mais que la clé reste identique, Streamlit considère que le contenu n'a pas évolué et conserve l'ancienne valeur en cache.

SOLUTION

Composer une clé unique par génération en intégrant un hash du contenu dans la clé du widget : `key=f"txt_{hash(data.get('texte','')) % 100000}_{name}"`. Avec cette clé dynamique, Streamlit reconstruit le widget à chaque génération et affiche le bon contenu.

```
st.text_area(
    f'Texte {name}',
    data.get('texte', ''),
    height=240,
    key=f"txt_{hash(data.get('texte','')) % 100000}_{name}",
)
```

🔗 Apprentissage : Les frameworks UI déclaratifs basent leur réactivité sur les clés. Toujours associer une clé qui change avec le contenu, pas une clé statique.

2.2. Refactor de l'architecture CSS modulaire

PROBLÈME

Au cinquième module CSS ajouté, le bloc injecté via `st.markdown` dépassait 800 lignes en chaîne unique. Toute modification ciblée devenait pénible et risquée. L'ajout d'une nouvelle fonctionnalité visuelle obligeait à parcourir l'ensemble du fichier pour trouver le bon endroit.

Cause identifiée :

Le code avait été initialement écrit avec une approche monolithique : une seule grande variable `GLOBAL_CSS` contenant des centaines de règles. La cohabitation de styles pour différentes pages rendait toute modification dangereuse pour les autres pages.

SOLUTION

Refactor complet en architecture modulaire : chaque domaine visuel (sidebar, formulaire, cartes plateformes, animations, etc.) devient une variable Python séparée (par exemple `_CSS_SIDEBAR`, `_CSS_GENERATOR_FORM`). La chaîne finale `GLOBAL_CSS` est construite par concaténation à l'initialisation. Au total, 18 modules CSS indépendants.

🔗 Apprentissage : *Diviser pour mieux régner. Une architecture modulaire, même triviale techniquement (concaténation de chaînes), démultiplie la productivité et réduit drastiquement le risque de régression.*

2.3. Corruption de fichiers Python lourds par OneDrive

PROBLÈME

À deux reprises, des écritures Python lourdes (le fichier `app.py` de plus de 2000 lignes) ont été tronquées en milieu de chaîne par OneDrive, cassant la compilation du fichier. Le bug était particulièrement frustrant car l'éditeur affichait le fichier complet, mais le disque contenait une version coupée à mi-mot.

Cause identifiée :

OneDrive verrouille temporairement les fichiers pendant l'indexation, ce qui peut interrompre les écritures longues. Le phénomène est connu mais peu documenté, et touche surtout les fichiers édités par des outils en ligne de commande (comme un éditeur Python ou un assistant IA).

SOLUTION

Adoption d'un workflow d'écriture atomique : toute écriture massive est effectuée d'abord dans le dossier `/tmp` du sous-système Linux (non synchronisé), puis copiée vers OneDrive en une seule opération atomique avec `cp`. Cette pratique a complètement éliminé le problème.

🔗 Apprentissage : *Méfiance envers les systèmes de synchronisation automatique pour les projets de développement. Préférer un dossier local non synchronisé pour les fichiers source, et utiliser Git pour la sauvegarde.*

2.4. Responsive design pour 3 tailles d'écran

PROBLÈME

L'interface, parfaitement rendue en desktop 1920×1080, devenait illisible sur tablette et inutilisable sur mobile : sidebar qui débordait, boutons trop petits pour le tactile, cartes résultats qui s'empilaient mal, onglets de plateformes qui sortaient de l'écran.

Cause identifiée :

Streamlit n'inclut pas de système de breakpoints natif. Le code initial assumait implicitement un écran large et utilisait des largeurs fixes en pixels.

SOLUTION

Ajout d'un module CSS dédié au responsive avec trois media queries : 1024px (tablette paysage), 768px (tablette portrait), 640px (mobile). Adaptation du padding, du nombre de colonnes, de la taille des polices, de la hauteur minimale des boutons (48px en mobile pour les guidelines tactiles), et de la sidebar qui se réduit à 280px.

🔗 Apprentissage : *Le responsive design n'est pas une option mais une nécessité. Tester systématiquement sur trois résolutions clés via les DevTools du navigateur (Ctrl+Shift+M en Chrome/Edge).*

Chapitre 3 — Défis d'orchestration n8n

Le workflow n8n constitue la pièce maîtresse de l'orchestration de publication. Sa configuration et sa mise en production ont soulevé trois défis distincts.

3.1. Mode test vs mode production de n8n et erreur 404

PROBLÈME

Le passage de n8n en mode production a réservé une surprise. En mode test (mode par défaut), le webhook accepte les requêtes uniquement après avoir cliqué sur Listen for Test Event dans l'interface, ce qui le rend inutilisable pour une intégration applicative. En basculant en mode production via le toggle de l'interface, le webhook continuait pourtant à retourner un 404.

Cause identifiée :

Le mode production exige deux actions distinctes : (1) basculer le toggle du workflow en Active, et (2) cliquer explicitement sur le bouton Publish situé en haut à droite de l'éditeur. Sans ce second clic, n8n retourne 404 même si le workflow apparaît visuellement comme actif.

SOLUTION

Documentation interne ajoutée dans le README et dans le runbook PowerShell : la séquence exacte à suivre pour activer un workflow en production est explicitée. Le test du webhook via Invoke-WebRequest depuis PowerShell permet de valider rapidement que le passage en production a réussi.

```
# Test du webhook en production
Invoke-WebRequest `
  -Uri 'https://primary-production-96d7d.up.railway.app/webhook/smartcontent-publish' `
  -Method POST `
  -Body '{"test":true}' `
  -ContentType 'application/json' `
  -UseBasicParsing | Select-Object StatusCode
```

🔗 Apprentissage : Lire les indices visuels d'une interface ne suffit pas toujours. Toujours valider techniquement avec un test côté API que l'état affiché correspond bien à l'état réel.

3.2. Routage Switch vers 12 publishers parallèles

PROBLÈME

Le workflow initial routait les 12 plateformes vers un nœud unique qui faisait du if/else en JavaScript. Cette approche fonctionnait mais ne profitait pas du parallélisme natif de n8n et était difficile à déboguer : un plantage sur une plateforme bloquait toutes les autres.

Cause identifiée :

n8n permet de paralléliser des branches via le nœud Switch en mode Rules. Chaque sortie du Switch correspond à une condition (par exemple `plateforme == 'linkedin'`) et déclenche une branche indépendante du graphe.

SOLUTION

Le nœud Switch a été configuré en mode Rules avec 12 outputs distincts, chacun connecté à un nœud Function dédié (Publier LinkedIn, Publier Instagram, etc.). Le nœud Merge en mode Append collecte ensuite les 12 résultats et les envoie au nœud Respond to Webhook.

🔗 Apprentissage : *Les outils no-code comme n8n offrent des constructions natives (Switch, Merge, parallélisme) qu'il faut savoir exploiter. Ne pas tout faire en code JavaScript embarqué — perdre la lisibilité visuelle annule l'intérêt principal de la plateforme.*

3.3. Migration n8n local vers n8n cloud sur Railway

PROBLÈME

L'application initialement développée s'appuyait sur n8n Desktop tournant sur la machine de l'auteur (port 5678). Cette configuration interdisait toute démonstration à distance et imposait que l'auteur laisse son ordinateur allumé en permanence. Pour le passage en production cloud, il a fallu migrer l'instance n8n vers un service hébergé.

Cause identifiée :

n8n Desktop ne propose pas de fonction d'export-import natif vers une instance distante. Le déploiement complet de n8n exige aussi un Postgres pour persister les workflows et idéalement un Redis pour gérer la file de messages des workers.

SOLUTION

Déploiement du template officiel n8n disponible sur Railway, qui inclut quatre services configurés : n8n principal, n8n worker, PostgreSQL, Redis. Le workflow JSON a été exporté depuis n8n Desktop et importé dans n8n cloud via copier-coller direct dans le canvas. L'URL du webhook de production a ensuite été mise à jour dans la variable `N8N_WEBHOOK_URL` du backend FastAPI et dans les Secrets de Streamlit Cloud.

🔗 Apprentissage : *Les templates officiels des plateformes cloud sont précieux : ils encapsulent les meilleures pratiques (Postgres + Redis pour n8n) qu'on aurait mis des heures à découvrir seul.*

Chapitre 4 — Défis du déploiement cloud

Le passage en production cloud, intervenu en semaine 7, a constitué la phase la plus dense en défis techniques. Six défis principaux ont jalonné cette étape : configuration Railway, healthcheck, CORS multi-environnements, gestion des variables d'environnement, migration n8n, et hébergement Streamlit Cloud.

4.1. Variables d'environnement manquantes au premier déploiement Railway

PROBLÈME

Lors du premier déploiement du backend FastAPI sur Railway, le service a échoué au healthcheck avec un message « Healthcheck failure ». Les logs montraient que l'application plantait dès le démarrage avec une exception Pydantic.

Cause identifiée :

pydantic-settings instancie l'objet Settings au moment de l'import du module config.py. Si les variables requises (openai_api_key, supabase_url, etc.) ne sont pas définies dans l'environnement, l'instantiation lève une ValidationError qui empêche uvicorn de démarrer. Au premier déploiement, les variables n'avaient pas encore été configurées dans l'interface Railway.

SOLUTION

Configuration de toutes les variables d'environnement (OPENAI_API_KEY, SUPABASE_URL, SUPABASE_KEY, SUPABASE_SECRET, SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, N8N_WEBHOOK_URL, APP_ENV, ALLOWED_ORIGINS, RESEND_API_KEY) via le Raw Editor de Railway, en copiant le contenu du .env local. Railway redéploie automatiquement après chaque modification de variable.

🔗 Apprentissage : *Toujours configurer les variables d'environnement AVANT de déclencher le premier déploiement. L'agent Railway peut donner un message d'erreur trompeur si l'application plante au démarrage.*

4.2. Healthcheck timeout sur Railway

PROBLÈME

Même après configuration des variables, le service échouait au healthcheck pendant les 30 premières secondes du déploiement. Railway considérait le service comme non opérationnel et tentait des redémarrages.

Cause identifiée :

L'application FastAPI met quelques secondes à démarrer (chargement des modules, vérification des connexions Supabase et OpenAI). Le healthcheck Railway frappait /health avant que uvicorn n'ait fini son boot, et la première requête retournait un 502.

SOLUTION

Configuration explicite du healthcheck dans le fichier railway.json avec healthcheckTimeout: 30 (au lieu de la valeur par défaut plus courte). La route GET /health du backend a été testée pour répondre rapidement (sans charger Supabase ni OpenAI au démarrage).

```
{
  "$schema": "https://railway.com/railway.schema.json",
  "build": { "builder": "NIXPACKS" },
  "deploy": {
    "startCommand": "uvicorn backend.main:app --host 0.0.0.0 --port $PORT",
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3,
    "healthcheckPath": "/health",
    "healthcheckTimeout": 30
  }
}
```

🔗 Apprentissage : *Toujours dimensionner les timeouts à la latence réelle de démarrage de l'application, pas à des valeurs théoriques.*

4.3. Configuration CORS multi-environnement**PROBLÈME**

Le passage en production a nécessité une refonte de la gestion CORS. En développement local, seules les origines localhost:8501 et localhost:3000 étaient autorisées. En production, le frontend Streamlit Cloud est hébergé sur un sous-domaine *.streamlit.app dont le nom exact n'est pas connu à l'avance par le backend.

Cause identifiée :

Streamlit Cloud génère le sous-domaine de l'application à partir du nom du repo et du compte GitHub, ce qui peut varier. Il n'est donc pas possible de hardcoder l'origine attendue dans la liste blanche CORS. Par ailleurs, ouvrir totalement le CORS avec ['*'] n'est pas acceptable pour la sécurité.

SOLUTION

La solution adoptée combine deux mécanismes : une liste statique d'origines lue depuis la variable d'environnement ALLOWED_ORIGINS, et un pattern regex (allow_origin_regex) qui autorise tout sous-domaine .streamlit.app. Cette approche est sécuritaire (seuls les sous-domaines Streamlit Cloud sont autorisés) et flexible (l'auteur peut ajouter des domaines custom via la variable d'env sans modifier le code).

```
import os
from fastapi.middleware.cors import CORSMiddleware
```

```

_default_origins = [
    'http://localhost:8501',
    'http://127.0.0.1:8501',
    'http://localhost:3000',
]
_env_origins = [o.strip() for o in
    os.environ.get('ALLOWED_ORIGINS', '').split(',') if o.strip()]

app.add_middleware(
    CORSMiddleware,
    allow_origins=_default_origins + _env_origins,
    allow_origin_regex=r'https://.*\.streamlit\.app',
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

```

🔗 Apprentissage : CORS est une question de balance entre sécurité et flexibilité. Les regex sur les sous-domaines de plateformes connues offrent un excellent compromis quand les noms exacts ne sont pas prédictibles.

4.4. Port dynamique sur Railway

PROBLÈME

Railway injecte un port différent à chaque redéploiement via la variable d'environnement PORT. Le code initial du backend, qui hardcodait port=8001 dans uvicorn.run(), refusait de démarrer correctement en production.

Cause identifiée :

L'application doit lire le port depuis la variable d'environnement plutôt que de le hardcoder. C'est une pratique standard pour les plateformes PaaS comme Railway, Heroku, Render, etc.

SOLUTION

Création d'un Procfile à la racine du projet qui contient la commande de lancement avec interpolation de la variable PORT : web: uvicorn backend.main:app --host 0.0.0.0 --port \$PORT. Le main.py de l'application a aussi été modifié pour lire PORT depuis l'environnement quand exécuté localement.

```

# Procfile à la racine du projet
web: uvicorn backend.main:app --host 0.0.0.0 --port $PORT

# Dans backend/main.py (pour usage local)
if __name__ == '__main__':
    import uvicorn
    port = int(os.environ.get('PORT', settings.app_port))
    uvicorn.run('backend.main:app', host='0.0.0.0', port=port, reload=True)

```

🔗 Apprentissage : Les PaaS modernes injectent dynamiquement la configuration via des variables d'environnement. Toujours rendre l'application configurable plutôt que hardcoder.

4.5. Gestion des Secrets sur Streamlit Cloud

PROBLÈME

Le frontend Streamlit a besoin de connaître l'URL du backend Railway et l'URL du webhook n8n pour fonctionner. En développement local, ces valeurs sont lues depuis le .env. En production, le .env n'est pas déployé (et ne devrait jamais l'être pour des raisons de sécurité).

Cause identifiée :

Streamlit Cloud expose un mécanisme natif de gestion des Secrets via l'onglet Settings de chaque application. Les valeurs définies sont exposées à l'application au runtime comme variables d'environnement.

SOLUTION

Configuration de deux secrets dans Streamlit Cloud au format TOML : BACKEND_URL pointant vers l'URL Railway du backend, et N8N_WEBHOOK_URL pointant vers le webhook de production n8n. Le code frontend lit ces valeurs au démarrage et bascule automatiquement entre développement local (lecture du .env) et production (lecture des Secrets).

```
# Secrets Streamlit Cloud (format TOML)
BACKEND_URL = "https://web-production-71b032.up.railway.app"
N8N_WEBHOOK_URL =
"https://primary-production-96d7d.up.railway.app/webhook/smartcontent-publish"
```

🔗 Apprentissage : Les plateformes cloud modernes offrent toutes un mécanisme de Secrets. L'utiliser dès le départ évite tout risque de fuite accidentelle de clés sensibles dans un dépôt Git.

4.6. Compatibilité Python 3.14 vs 3.12 (Railway)

PROBLÈME

Le projet a été développé en local avec Python 3.14, qui présente certaines spécificités (notamment l'exclusion de Pillow et MoviePy). Au déploiement sur Railway, le build échouait avec une erreur de version Python introuvable.

Cause identifiée :

Railway ne supportait pas encore Python 3.14 au moment du déploiement. La plateforme propose des versions stables jusqu'à 3.12.7. Cette divergence aurait pu introduire des bugs subtils si certaines features 3.14 avaient été utilisées.

SOLUTION

Création d'un fichier `runtime.txt` à la racine du projet contenant `python-3.12.7`, qui force Railway à utiliser cette version pour le build. Le code Python a été audité pour s'assurer qu'aucune syntaxe 3.13+ exclusive n'était utilisée. Toutes les fonctionnalités utilisées (FastAPI, Pydantic, OpenAI, Supabase) sont compatibles 3.12.

🔗 Apprentissage : *Toujours documenter explicitement la version Python cible dans un fichier `runtime.txt` ou `.python-version`. Les hypothèses sur la version par défaut d'une plateforme peuvent causer des heures de débogage.*

Chapitre 5 — Défis d'intégration email (Resend)

L'ajout de la fonctionnalité de livraison email a transformé le workflow simulé en livrable concret pour l'utilisateur. Quatre défis ont marqué cette phase : choix du service, conception du template, gestion des images inline, et délivrabilité.

5.1. Choix du service email entre quatre options

PROBLÈME

Une fois la décision prise d'envoyer la campagne par email, il restait à choisir un service tiers. Quatre options se présentaient : Resend, SendGrid, Mailgun, AWS Simple Email Service. Chacun avec son SDK, son free tier, son setup, sa délivrabilité. Le choix devait être tranché rapidement pour ne pas bloquer la suite du développement.

Cause identifiée :

Les critères de choix étaient multiples et parfois contradictoires : simplicité du SDK, générosité du free tier, qualité du dashboard, rapidité du setup, et qualité de la délivrabilité (taux d'arrivée en boîte de réception principale).

SOLUTION

Resend a été retenu pour quatre raisons cumulées : SDK Python minimaliste (deux lignes pour envoyer un email), free tier de 3000 emails par mois (largement suffisant pour le MVP académique), dashboard moderne avec tracking en temps réel des emails (delivered, opened, clicked, bounced), et inscription via GitHub utilisable en cinq minutes sans validation manuelle préalable.

🔗 Apprentissage : *Quand on choisit une dépendance pour un projet à délais courts, privilégier la vitesse d'onboarding plutôt que la richesse fonctionnelle. Un service moins puissant mais qu'on maîtrise en 30 minutes vaut mieux qu'un service complet qu'on bricole encore après deux jours.*

5.2. Conception d'un template HTML compatible multi-clients mail

PROBLÈME

Le premier template HTML envoyé via Resend s'affichait correctement dans Gmail mais cassé dans Outlook desktop, partiellement dans Yahoo Mail, et illisible dans Apple Mail. Les couleurs ne s'appliquaient pas, les marges étaient erratiques, les images mal alignées.

Cause identifiée :

Les clients mail anciens (notamment Outlook desktop sur Windows) supportent une fraction très limitée du CSS moderne. Pas de Flexbox, pas de Grid, pas de variables CSS, support partiel des sélecteurs. Le rendu dépend du moteur de rendu utilisé (Word pour Outlook, WebKit pour Apple Mail).

SOLUTION

Réécriture complète du template en HTML pur avec CSS inline sur chaque élément (`style="..."`). Aucune classe CSS, aucune feuille de style externe. Les couleurs hexadécimales sont littérales, les marges et paddings exprimés en pixels. Le layout utilise des tables HTML traditionnelles pour les zones complexes (compatibilité maximum).

🔗 Apprentissage : *Les emails HTML restent un domaine technologique à part. Tester sur les 4 principaux clients (Gmail, Outlook, Yahoo, Apple) AVANT de finaliser le template.*

5.3. Récupération sécurisée de l'email du user depuis Supabase

PROBLÈME

Quand l'utilisateur clique sur le bouton « Recevoir par email », le frontend ne connaît pas directement son adresse email — il ne stocke que le `user_id` dans la session. Comment récupérer l'email d'inscription côté backend, sans le passer par le frontend (ce qui exposerait à une manipulation client) ?

Cause identifiée :

Le pattern naïf consisterait à envoyer l'email depuis le frontend (récupéré au login), mais cela permettrait à un utilisateur malveillant de modifier l'email pour envoyer la campagne à n'importe qui. Le backend doit donc faire la résolution lui-même.

SOLUTION

La route POST `/generate/send-email` reçoit le `user_id` du frontend (déjà validé via le JWT), puis effectue une requête Supabase `SELECT email FROM users WHERE id = user_id` avec la clé `service_role`. L'adresse email récupérée côté serveur est utilisée comme destination, sans jamais faire confiance à une valeur fournie par le frontend.

```
@router.post('/send-email', response_model=SendEmailResponse)
def send_email_route(payload: SendEmailRequest):
    # Récupérer l'email depuis Supabase via le user_id
    res = supabase_admin.table('users').select('email')
        .eq('id', payload.user_id).execute()
    if not res.data:
        raise HTTPException(404, 'Utilisateur introuvable.')
    to_email = res.data[0].get('email')
    # Envoyer via Resend
    result = send_campaign_email(to_email, payload.sujet, ...)
```

🔗 Apprentissage : *Ne jamais faire confiance à des données qui transitent par le frontend pour des opérations critiques. Toujours résoudre les identifiants en données réelles côté backend, à partir d'une source de vérité (la base de données).*

5.4. Premier email arrivé en spam

PROBLÈME

Le tout premier email envoyé en test via Resend est arrivé directement dans le dossier Spam de Gmail. Le sujet, le contenu et l'expéditeur étaient tous légitimes. Pourtant, Gmail a marqué le message comme suspect.

Cause identifiée :

L'expéditeur configuré était le domaine de test fourni par Resend (onboarding@resend.dev). Ce domaine est partagé entre tous les nouveaux utilisateurs de Resend qui n'ont pas encore vérifié leur domaine custom. Gmail considère ce domaine partagé comme suspect par défaut, surtout pour un destinataire qui n'avait jamais reçu d'email de ce domaine auparavant.

SOLUTION

Acceptation de cette limitation pour le MVP académique : les emails de test arrivent parfois en spam mais la fonctionnalité est techniquement opérationnelle. Pour la phase commerciale, l'étape suivante consistera à vérifier un domaine custom dans Resend en ajoutant trois enregistrements DNS (SPF, DKIM, DMARC) au registrar du domaine. Cette opération prend environ cinq minutes et améliore drastiquement la délivrabilité au-delà de 95 % en boîte de réception principale.

🔗 Apprentissage : *La délivrabilité email n'est pas une question de code mais de réputation de domaine. Pour atteindre une délivrabilité élevée, il faut un domaine custom correctement authentifié (SPF, DKIM, DMARC) et un volume d'envoi régulier qui établit la réputation.*

Chapitre 6 — Défis méthodologiques

Au-delà des défis purement techniques, trois défis méthodologiques ont marqué le projet : la gestion d'un dépôt Git avec des artefacts lourds, la documentation parallèle au développement, et la coordination des nombreux livrables soutenance.

6.1. Squash de l'historique Git pour un commit unique propre

PROBLÈME

Après plusieurs semaines de développement, le dépôt GitHub contenait quatre commits anciens (b6a104c, 54278cc, b8733a7, WIP avant squash) avec des messages de qualité variable. L'auteur souhaitait présenter au jury un historique propre avec un seul commit final intitulé « SmartContent AI v2 - Version commerciale ».

Cause identifiée :

Le mode opératoire git classique (git reset puis recommit) ne fonctionne pas directement pour effacer l'historique distant. La commande git push --force était nécessaire mais devait être maniée avec précaution pour ne pas perdre de code.

SOLUTION

Application d'une séquence Git avancée : git rev-parse "HEAD^{tree}" pour récupérer le SHA du tree actuel, git commit-tree avec un nouveau message pour créer un commit orphelin, git reset --hard sur le nouveau SHA, puis git push --force pour propager. Le résultat : un historique distant unique sur main avec le bon message. Note : sur PowerShell, le caractère ^ dans HEAD^{tree} doit être encadré de guillemets doubles pour éviter d'être interprété comme escape character.

```
# Squash en un commit propre
$tree = git rev-parse "HEAD^{tree}"
$newSha = git commit-tree $tree -m "SmartContent AI v2"
git reset --hard $newSha
git push -u origin main --force
```

🔗 Apprentissage : Les opérations Git avancées (rebase interactif, commit-tree, force push) sont puissantes mais dangereuses. Toujours tester sur une branche de sauvegarde avant d'appliquer sur main.

6.2. Protection des secrets dans le dépôt Git

PROBLÈME

Le projet contient un fichier .env avec des secrets très sensibles : clé API OpenAI, clé service-role Supabase, secret JWT, clé Resend. Une fuite de ce fichier dans le dépôt GitHub public aurait des conséquences immédiates : utilisation frauduleuse du compte OpenAI (potentiellement des milliers de dollars), accès

complet aux données utilisateurs Supabase, possibilité d'usurper l'identité de tous les utilisateurs via JWT forgés.

Cause identifiée :

Par défaut, git add . ajoute tous les fichiers non-ignorés. Sans .gitignore correctement configuré, le .env aurait été committé dès le premier push.

SOLUTION

Le fichier .gitignore a été configuré dès le premier jour avec deux règles essentielles : .env (exclut le fichier de secrets) et !.env.example (autorise le template public). Avant chaque commit important, vérification via git ls-files | grep .env qui ne doit RIEN retourner. En cas de fuite accidentelle, un protocole de rotation des clés en moins de 30 minutes a été préparé (régénération OpenAI, reset Supabase service-role key, nouvelle clé Resend).

```
# Contenu du .gitignore
.env
.env.*
!.env.example
venv/
__pycache__/
*.pyc

# Vérif AVANT chaque commit :
git ls-files | Select-String -Pattern '\.env$'
# (NE doit RIEN retourner)
```

🔗 Apprentissage : La sécurité des secrets ne se rattrape pas. Mieux vaut configurer .gitignore correctement dès le départ et vérifier à chaque commit, plutôt que de découvrir une fuite après plusieurs jours.

6.3. Coordination des livrables soutenance

PROBLÈME

Le projet inclut une quantité importante de livrables : mémoire écrit (52 pages initialement, 42 après refonte V2), deck soutenance (20 slides puis 21), pitch deck initial (12 slides), guide vidéo démo (10 pages), récap final (14 pages), runbook PowerShell, addendum cloud. Au total plus de 100 pages de documentation à maintenir cohérentes entre elles, dans un contexte où le projet lui-même évoluait quotidiennement (ajout déploiement cloud, ajout email Resend).

Cause identifiée :

Chaque modification fonctionnelle du projet (nouveau service, nouveau bouton, nouvelle URL) impactait potentiellement plusieurs documents. Sans process clair, des incohérences sont apparues : le mémoire

mentionnait localhost alors que l'app tournait sur Railway, le deck citait 5 services au lieu de 6, le guide vidéo demandait de lancer uvicorn en local alors que tout était cloud.

SOLUTION

Adoption d'une organisation en deux sous-dossiers : docs/livrables/FINAL/ pour les versions à jour à présenter au jury, et docs/livrables/ARCHIVE/ pour les anciennes versions conservées. Création d'un README dans chaque dossier qui explicite quel fichier utiliser pour quoi. Audit régulier des documents via grep sur les mots-clés cloud/email pour identifier ceux qui ne sont plus à jour. Régénération synchronisée des versions V2 quand une évolution majeure du projet rendait obsolètes plusieurs documents.

🔗 Apprentissage : *Quand un projet produit beaucoup de documentation, il faut un mécanisme explicite pour distinguer « ce qui est à jour » de « ce qui est en attente de mise à jour ». Sinon, le risque de présenter un document obsolète au jury est élevé.*

Bilan — Apprentissages transférables

Au-delà des solutions techniques individuelles, les 24 défis rencontrés ont permis de consolider un ensemble d'apprentissages transférables à tout projet d'application web moderne. Ces apprentissages sont synthétisés ci-dessous, organisés par grandes thématiques.

Architecture et choix de stack

- Préférer les bibliothèques fines (bcrypt direct) aux wrappers complexes (passlib) pour les fonctionnalités critiques de sécurité.
- Lire la documentation officielle d'une bibliothèque avant de coder, plutôt que de se baser sur des tutoriels potentiellement obsolètes.
- Privilégier les opérations parallèles (ThreadPoolExecutor) pour les tâches I/O-bound qui appellent des API tierces.
- Utiliser les modes natifs des API (response_format JSON) plutôt que de coder du parsing fragile en aval.

Sécurité applicative

- Configurer .gitignore correctement dès le premier commit pour protéger les secrets.
- Vérifier systématiquement avant chaque commit que les fichiers sensibles ne sont pas dans le staging.
- Ne jamais faire confiance aux données fournies par le frontend pour des opérations critiques (toujours résoudre côté backend depuis la source de vérité).
- Préparer un protocole de rotation des clés en cas de fuite accidentelle.

Déploiement cloud et configuration

- Toujours rendre l'application configurable via variables d'environnement, jamais hardcoder des valeurs.
- Documenter explicitement la version Python cible (runtime.txt) plutôt que de se fier à la version par défaut de la plateforme.
- Configurer les variables d'environnement AVANT de déclencher le premier déploiement.
- Dimensionner les timeouts de healthcheck à la latence réelle de démarrage, pas à des valeurs théoriques.
- Pour CORS, combiner une liste statique d'origines et un pattern regex pour les sous-domaines de plateformes cloud.

Frontend et UX

- Toujours utiliser des clés dynamiques pour les widgets de framework déclaratifs (Streamlit, React) afin d'éviter le cache obsolète.
- Adopter une architecture modulaire dès qu'un fichier dépasse quelques centaines de lignes — la dette technique se paye cher ensuite.
- Tester systématiquement le responsive sur trois résolutions (desktop, tablette, mobile).
- Concevoir les templates email en HTML inline avec tables pour la compatibilité multi-clients.

Outils et méthodologie

- Pour les fichiers source, préférer un dossier local non synchronisé (OneDrive, Dropbox) à cause des risques de corruption.
- Workflow d'écriture atomique : écrire dans /tmp puis copier vers la destination finale.
- Lire les indices visuels d'une interface ne suffit pas toujours : toujours valider techniquement avec un test API.
- Quand un projet produit beaucoup de documentation, organiser explicitement les versions à jour vs obsolètes (dossiers FINAL et ARCHIVE).
- Privilégier la vitesse d'onboarding d'un service lors d'un projet à délais courts, plutôt que la richesse fonctionnelle.

Conclusion

Ce journal des défis et solutions rend visible la part importante du travail consacrée non pas à l'écriture du code productif, mais à la résolution des obstacles qui surviennent inévitablement dans un projet réel. Vingt-quatre défis ont été identifiés, documentés et résolus au cours des sept semaines de développement. Pour chacun, une solution a été trouvée et un apprentissage en a été tiré.

Cette approche réflexive transforme ce qui aurait pu rester une simple suite d'incidents en un capital méthodologique transférable. Les patterns identifiés — préférer les bibliothèques fines aux wrappers, paralléliser les I/O, sécuriser les secrets dès le premier commit, configurer dynamiquement plutôt que hardcoder — sont applicables à n'importe quel projet d'application web moderne.

Le résultat final, l'application SmartContent AI déployée en production cloud avec livraison email automatique, n'aurait pas été possible sans cette discipline de résolution méthodique. Chaque défi surmonté représente une compétence acquise et une heure de débogage économisée sur le prochain projet.

Ce document accompagne le mémoire principal et le deck de soutenance. Il vise à donner au jury une vision complète et honnête du parcours réel du projet, depuis la première ligne de code écrite jusqu'au dernier email livré dans la boîte mail du testeur.